



(Integer Extended Infinitely ∞)

Arbitrary Precision Integer Library

Mohammad KazemiMoghaddam

Version 1.0

intEx8

(Integer Extended Infinitely ∞)

Arbitrary Precision Integer Library

Mohammad KazemiMoghaddam

Version 1.0

intEx8: Integer Extended Infinitely ∞

Overview

intEx8 is a C library providing functionality for **arbitrary-precision integer arithmetic**. It supports:

- Mathematical operations: `+`, `-`, `*`, `/`, `%`, `abs`, `negate`
- Bitwise operations: `~`, `&`, `|`, `^`, `<<`, `>>`
- Comparison Operations: `==`, `<`, `<=`, `>`, `>=`

Terminology

- **intx**: Represents an arbitrarily large integer.
 - **digit**: A 32-bit integer unit (`dig_t`) used to store digits of **intxs** internally.
 - **Base Representation**: Integers are stored as **arrays of 32-bit digits** in base 2^{32} .
 - **Sign Representation**: The **most significant bit** determines the sign (0 = positive, 1 = negative).
-

Interfaces in intEx8

intEx8 provides two interfaces:

1. `ix8_` Interface

- Allocates memory for results automatically.
- Developers must **free memory** using `ix8_free()`.
- Implemented in `ix8.c` and `ix8.h`.

2. `i8_` Interface

- Requires the caller to **allocate and manage memory**.
 - Offers better performance but requires manual memory management.
 - Implemented in `i8.c` and `i8.h`.
-

String Conversion Utilities

Convert Big Integer to Decimal String

```
char *ix8_to_string(intx_t x);
```

- Converts a big integer to a **decimal string representation**.
- The returned string **must be freed** using `ix8_free_string()`.

Initialize a Big Integer from a Decimal String

```
intx_t ix8_from_string(const char *str);
```

Example

```
intx_t x = ix8_from_string("290293826899404827630986747261039409382000000");  
...  
ix8_free(x);
```

- More initialization methods will be added in future versions.

Error Handling

- Both `ix8_` and `i8_` interfaces use `int intex8_errno` to store error codes.
- Error codes are defined in `intEx8.h`.

Using intEx8 in a Project

1. Add the following files to your project/build system:
 - `ix8.c`, `i8.c`, `toom3_multiply.c`
2. Include the required header file:
 - `#include "intEx8.h"` to call library functions.

Handling Large Integers in Multiplication and Division

Multiplication Limits

- The **maximum supported operand size** is limited (defined in `intEx8.h`) by `INTEX8_MAX_MULTIPLICATION_DIGITS`.
- This value **can be increased**, but increasing it also **increases the buffer size** for the Toom-3 algorithm implemented in `_toom3_multiply()`.
- `#define INTEX8_TOOM3_DYNAMIC_BUF` may also be used if

Division Limits

- The **dividend size** is limited by `INTEX8_MAX_DIVIDEND_DIGITS` (defined in `intEx8.h`). Developers **can increase** it if larger size for dividend is needed.
-

Best Practices for Using intEx8

✓ Always check `intex8_errno` after calling functions. If it is not `INTEX8_OK`, the return value is invalid.

✓ Avoid recursive function calls such as:

```
ix8_add(ix8_multiply(a, b), y); // ⚠ Memory leak!
```

✓ Avoid assigning results to the same variable:

```
x = ix8_add(x, y); // ⚠ Memory leak!
y = ix8_add(x, y);
```

Instead, store the result in a temporary variable and **free memory before reassignment**.

Advanced Topics

Memory Allocation Considerations

- The `ptr` member of `intx_t` may be allocated with **more digits than size**.
- Use utility functions to determine **actual allocated memory**.

Handling Sign Extension

- `intEx8` does not provide `unsigned intx_t`.
- If a **negative** integer needs to be treated as **positive**, add a leading 0 digit.

Example

```
x = {{1, 2, 2147483648}, 3}; // x is a negative intx
y = {{1, 2, 2147483648, 0}, 4}; // y is a positive intx equal to -x
```

Extreme Positive Integers

- If manually setting an **Extreme Positive Integer**, **allocate an extra digit** and set it to 0.

- Otherwise, operations like `i8_add_self(x, y)`, where `x` is an Extreme Positive and `y` is a positive integer, will cause a **memory overrun**.

Example

```
intx_t x = {{2147483647, 1073741824}, 2}; // x is an Extreme Positive
Integer, i.e. the maximum possible integer can be set to a 2-digit intx. Hex
representaion of x is 0x7FFF'FFFF'FFFF'FFFF.
```

Extreme Negative Integers

- If manually setting an **Extreme Negative Integer**, allocate an extra digit and set it to `0xFFFFFFFF`.
- Otherwise, operations like `ix8_negate_self()` may not work correctly and `INTEX8_ERR_CANNOT_NEGATE_EXTREME_NEGATIVE` will be set.

Example

```
intx_t x = {{0, 0, 2147483648}, 3}; // x is an Extreme Neative Integer, i.e.
the minimum possible integer can be set to a 3-digit intx. Hex representation
of x is 0x8000'0000'0000'0000.
```

Future Improvements

Bug Fixes – Any reported issues will be addressed.

C++ Interface – A C++ wrapper class for `i8_` functions with overloaded `+`, `-`, `*`, `/`, `%` operators.

Floating-Point Support – Arbitrary-precision floating-point arithmetic.

Fractional Number Support – Exact representation of rational numbers.

Performance Enhancements – Further optimizations for large-number calculations.

Conclusion

intEx8 is a **high-performance** arbitrary-precision integer library designed for **efficiency, flexibility, and scalability**. With its **two interface sets**, robust error handling, and powerful mathematical capabilities, it is an ideal choice for developers working with **large integers beyond fixed-size types**.